

- > Implantación de una app que gestiona incidencias conectándose a una BBDD embebida (SQLite).
- > Enunciado:

Contexto empresarial

La empresa TechSolutions S.L. necesita desarrollar un sistema interno para gestionar incidencias reportadas por sus empleados. Actualmente, el proceso se realiza por correo electrónico, lo que genera:

falta de seguimiento
duplicidad de incidencias
mala asignación de técnicos
ausencia de métricas

El equipo de desarrollo ha decidido construir una primera versión del backend en Python.

Tu equipo será responsable de desarrollar este sistema.

Objetivo del proyecto

Construir una aplicación backend en Python que permita:

registrar técnicos
crear incidencias (tickets)
asignar incidencias a técnicos
gestionar el estado de las incidencias
consultar información relevante

El sistema debe seguir buenas prácticas profesionales:

tipado en Python
acceso a base de datos
separación por capas
uso de control de versiones con Git y GitHub

Gestión de técnicos

El sistema debe permitir:

Registrar un técnico con:
nombre
email
estado activo/inactivo
Consultar todos los técnicos

Gestión de incidencias (tickets)

Cada ticket debe contener:

título
descripción
email del creador
estado:
OPEN
IN_PROGRESS

CLOSED

técnico asignado (opcional)

Operaciones sobre tickets

El sistema debe permitir:

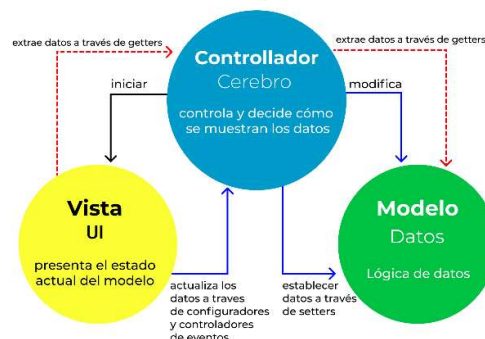
Crear un ticket
Asignar un ticket a un técnico
Cambiar el estado de un ticket
Cerrar un ticket
Consultar:
todos los tickets
tickets abiertos
Reglas de negocio (muy importante)

Debes implementar las siguientes validaciones:

No se puede crear un ticket con título vacío o demasiado corto
El email del creador debe ser válido (contener @)
No se puede asignar un ticket a un técnico inexistente
No se puede asignar un ticket a un técnico inactivo
No se puede cerrar un ticket ya cerrado
(Nivel medio) No se puede cerrar un ticket sin técnico asignado

- > **CRUD** = Create Read Update Delete
- > Tipado en Python → para hacer nuestra app más restrictiva.
- > **Modelo vista-controlador**: estrategia de desarrollo de programación.

Patrones de Arquitectura MVC



- > **Modelo** → los datos, defino el tipo de datos dentro de mi app.
- > **Controlador** → la lógica de la app.
- > **Vista** → lo que ve el usuario.

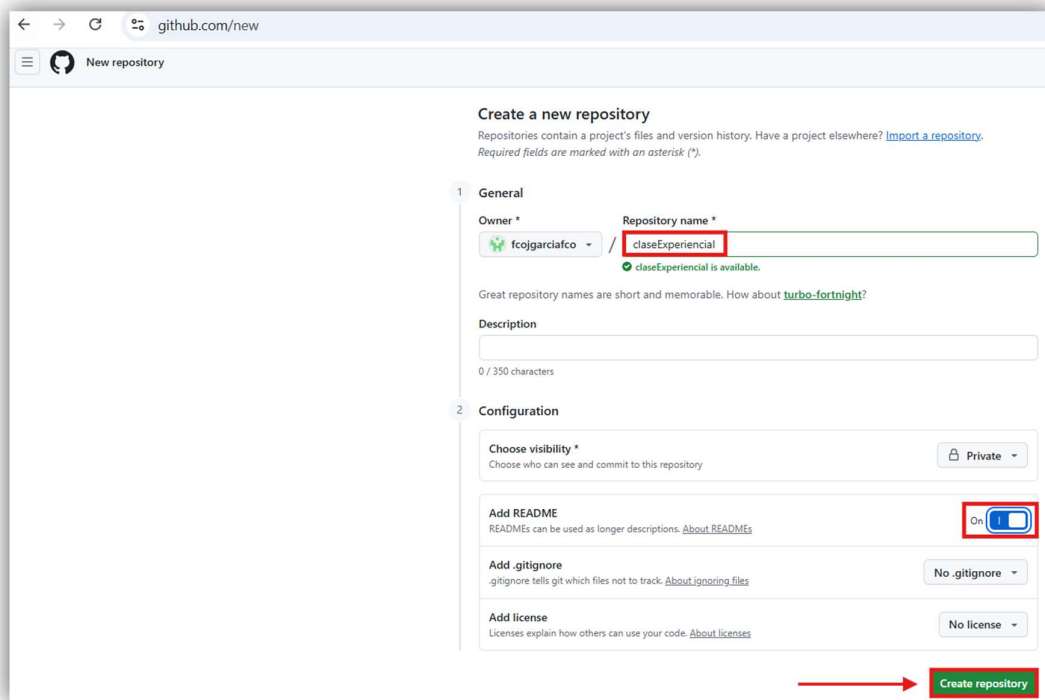
- > **Modelo** → archivos .py
- > **Controlador** → archivos .py
- > **Vista** → archivos .py

→ Mucha mayor fluidez dentro de la app → en caso de Big Data la carga será mucho más rápida; si muestro muchas imágenes en la app evito que se trabase.

> El código se divide en capas (archivos que se conectan entre si).

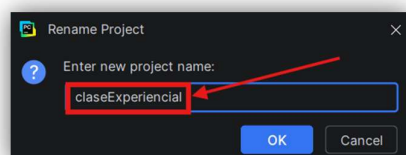
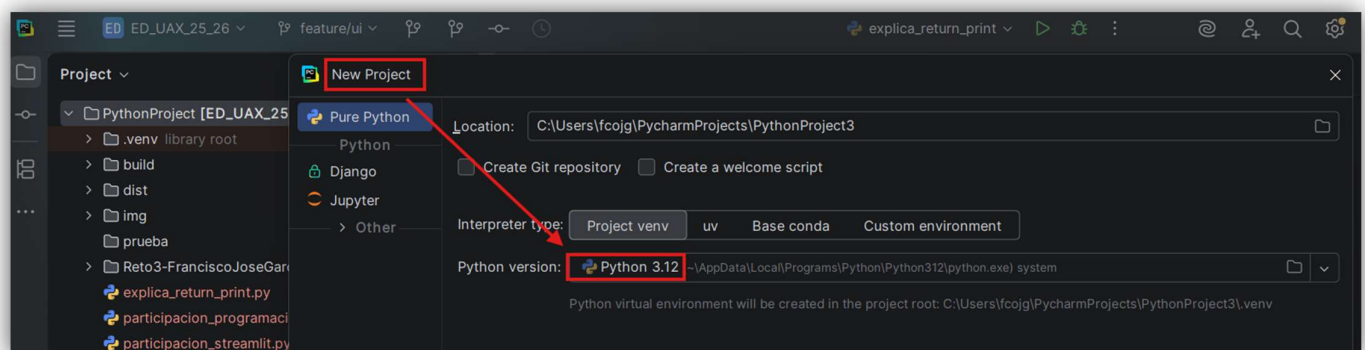
> Más fácil buscar errores si nos equivocamos en algo.

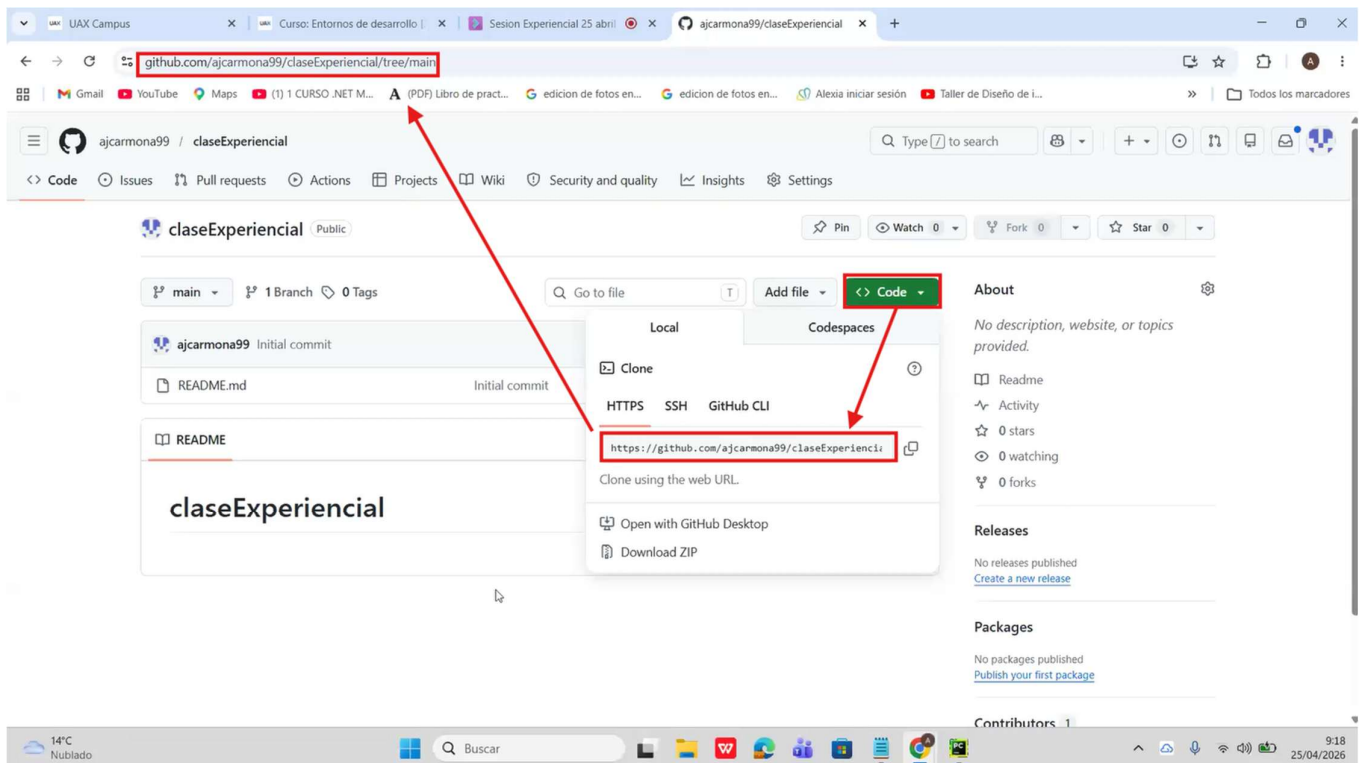
> GitHub → preparamos el repositorio para el proyecto.



> **Enlace repositorio profesor:**

<https://github.com/ajcarmona99/IncidenciaTickets>



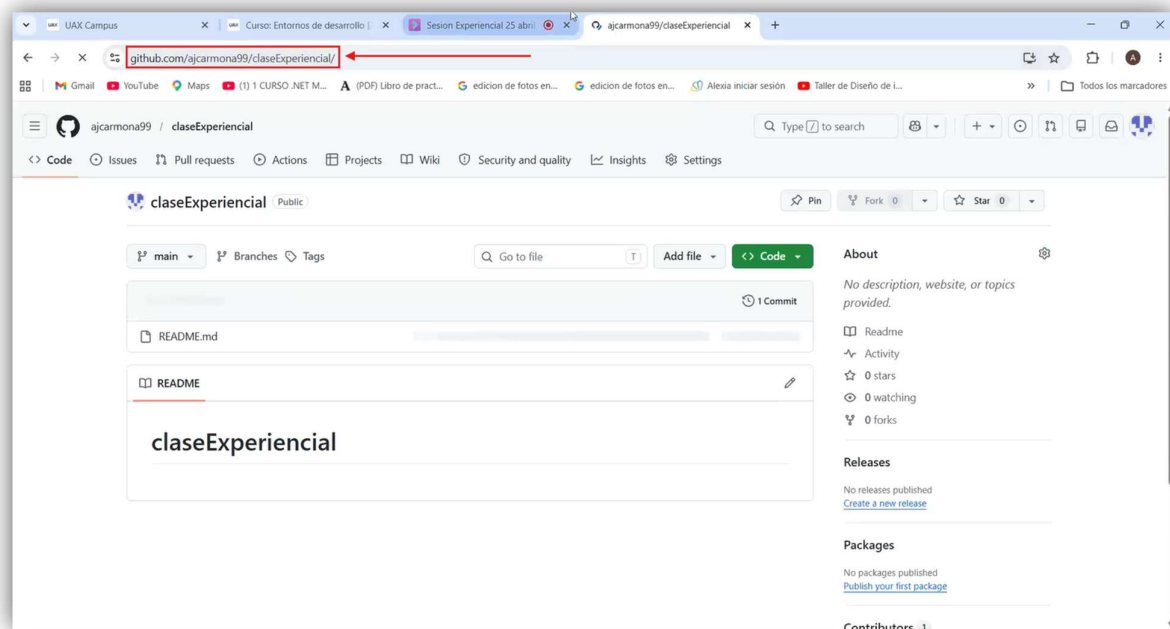


Estamos dentro de una rama.

Aconseja dejarlo en la raíz y asociar un alias a la URL.

`github.com/ajcarmona99/claseExperiencial/tree/main`

Aconseja quitar tree/main, para coger el código de la raíz, no de una rama.



> En la terminal lo primero que va a hacer es inicializar un repo de Git:

```
Terminal Local x + v

Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

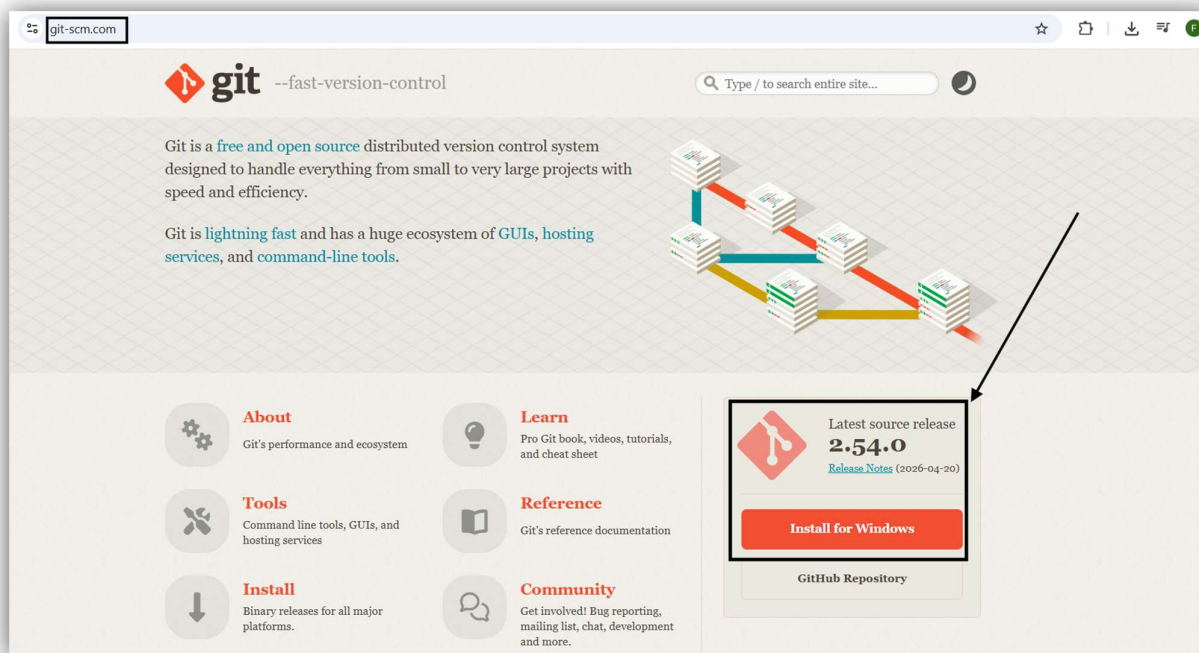
Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

(.venv) PS C:\Users\fcojg\PycharmProjects\PythonProject3> git init
Initialized empty Git repository in C:/Users/fcojg/PycharmProjects/PythonProject3/.git/

(.venv) PS C:\Users\fcojg\PycharmProjects\PythonProject3> 
```

> Los que no seamos Mac tenemos antes que instalar Git:

<https://git-scm.com/>



> Si no, no reconocerá los comandos git.

> No necesario en Mac o Linux.

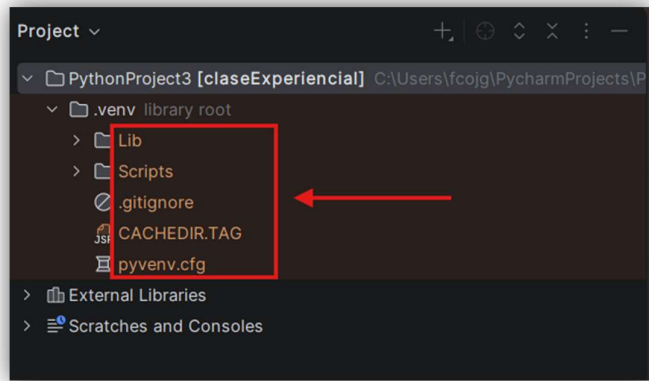
```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\fcojg> git --version
git version 2.54.0.windows.1
PS C:\Users\fcojg> 
```

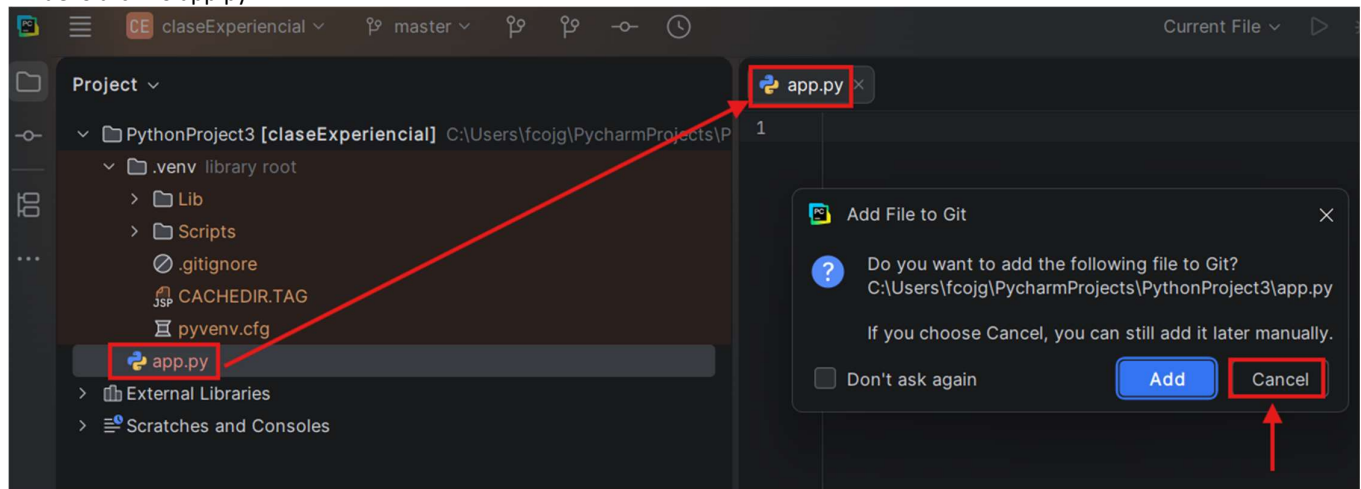
> Como ya teníamos instalado GitHub reconoce los comandos.

> Después de inicializar el git veremos archivos en naranja:



> Nos indica que estos están en memoria, pero aún no los ha añadido al repositorio de git.

> Nuevo archivo app.py.



> De primeras no lo añade al repo, lo hará luego por línea de comandos.

> Vamos a tener una serie de archivos:

- app.py → tendrá la app principal.
- db.py → que contendrá la BBDD.
- models.py → modelos.
- repository.py → repositorio con datos que usaremos.
- service.py → donde estará el CRUD.
- seed.py → un archivo de ejemplo, el primero tratamiento de datos; los primeros datos de ejemplo que meteré en la BBDD que está vacía de inicio.

→ Los relacionaremos empleando POO.

> models.py → vamos a tener todos los tipos de datos que vamos a definir:

```
from enum import Enum

# Tipos de estado del ticket
class TicketStatus(Enum):
    OPEN = "OPEN"
    IN_PROGRESS = "IN_PROGRESS"
    CLOSED = "CLOSED"
```

Esta pasando una variable que no está metida dentro de un string.

> Define las variables que va a almacenar en la BBDD:

```
from dataclasses import dataclass
from typing import Optional
```

```
@dataclass
class Technician:
    id: Optional[int]
```

Lo que estás haciendo con este bloque de código es definir la estructura inicial para representar a un técnico, utilizando algunas de las herramientas más modernas y eficientes que ofrece Python para la gestión de datos.

Básicamente, estás preparando el terreno para crear objetos que guarden información sobre técnicos de una manera limpia, tipada y fácil de mantener.

Aquí te explico al detalle qué hace cada línea:

1. Las Importaciones (Imports)

- `from enum import Enum`: Importas la clase base para crear enumeraciones. Aunque no la usas en el fragmento de código que mostraste, se nota que tu intención es definir tipos fijos más adelante (por ejemplo, el estado del técnico: `ACTIVO`, `INACTIVO`, o su especialidad: `ELECTRICISTA`, `MECÁNICO`).
- `from dataclasses import dataclass`: Importas un decorador que automatiza la creación de clases orientadas a almacenar datos.
- `from typing import Optional`: Importas un alias de tipado que indica que un valor puede ser de un tipo específico o puede ser `None` (vacío).

2. La estructura del Técnico (@dataclass)

En lugar de escribir una clase tradicional de Python donde tendrías que programar manualmente el método constructor `__init__`, el decorador `@dataclass` lo hace por ti tras bambalinas.

```
@dataclass
class Technician:
    id: Optional[int]
```

Al definir `id: Optional[int]`, estás diciendo que:

- Cada técnico tendrá un atributo llamado `id`.
- El tipado es flexible: Ese `id` debería ser un número entero (`int`), pero también es totalmente válido que sea `None` (por ejemplo, si el técnico es nuevo y aún no se ha guardado en una base de datos que le asigne un número).

¿Cómo se vería esto en la práctica?

Gracias a que usaste `@dataclass`, ya puedes interactuar con tu clase de esta forma sin haber programado nada más:

```
# Crear un técnico que ya tiene un ID asignado
tecnico_A = Technician(id=101)

# Crear un técnico nuevo que aún no tiene ID (por eso el Optional)
tecnico_B = Technician(id=None)

# Imprimir el objeto (dataclass te regala un formato limpio automáticamente)
print(tecnico_A) # Resultado: Technician(id=101)
```

```
from dataclasses import dataclass
from typing import Optional
```

```
@dataclass
class Technician:
    id: Optional[int]
    name: str
    email: str
    active: bool = True
```

> bool → por defecto True

> Estamos definiendo dos clases que serán tablas (tabla Technician y tabla Ticket → 2 entidades) con los atributos de las mismas que serán los campos de las tablas:

id será la clave primaria de cada entidad

technician_id → foránea

```
@dataclass
class Technician:
    id: Optional[int]
    name: str
    email: str
    active: bool = True

@dataclass
class Ticket:
    id: Optional[int]
    title: str
    description: str
    created_by: str
    status: TicketStatus
    technician_id: Optional[int] = None
```

> db.py → el que usará SQLite:

> Ya hay una librería dentro de Python base para usar SQLite:

```
import sqlite3
```

> La BBDD va a ir dentro del proyecto embebida → está dentro de la carpeta del proyecto a la altura del resto de archivos.

> También hay librerías para conexión a BBDD de Oracle, MySQL ...

```
import sqlite3
from sqlite3 import Connection

DB_NAME = "claseExperiencial.db"

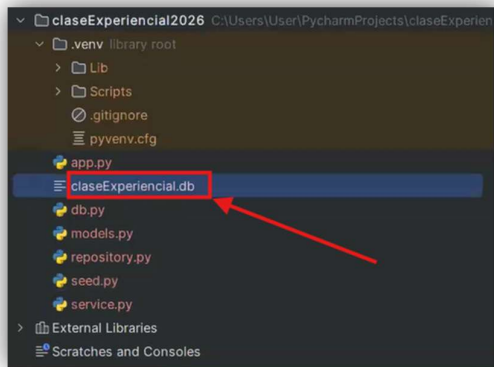
def get_connection() -> Connection: #Define el tipo de dato que está devolviendo la función
    conn = sqlite3.connect(DB_NAME)
    return conn

if __name__ == "__main__":
    get_connection()
```

Al ejecutar llama a la función que hemos creado.

Si la bd no existe la crea:

→



> En sqlite no pide usuario y contraseña ¿?? Al ser BBDD embebida → es diferente para cada usuario en su ordenador.

> No está en servidor, es en local.

> Ahora crea una tabla.

> CURSOR para hacer consulta a la BBDD →

```
def init_db() -> None:
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    CREATE TABLE technicians (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        email TEXT NOT NULL UNIQUE,
        active INTEGER NOT NULL CHECK (active IN (0,1))
    ) """)
```

```
def init_db() -> None:
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    CREATE TABLE technicians (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        email TEXT NOT NULL UNIQUE,
        active INTEGER NOT NULL CHECK (active IN (0,1))
    ) """)

    cursor.execute("""
    CREATE TABLE tickets (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        description TEXT NOT NULL,
        created_by TEXT NOT NULL,
        status TEXT NOT NULL,
        technician_id INTEGER,
        FOREIGN KEY (technician_id) REFERENCES technicians(id)
    ) """)
```

> Para que los cambios en la BBDD queden permanentes → COMMIT: como un Ctrl + S


```
def init_db() -> None: 1 usage
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    CREATE TABLE technicians (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        email TEXT NOT NULL UNIQUE,
        active INTEGER NOT NULL CHECK (active IN (0,1))
    ) """)

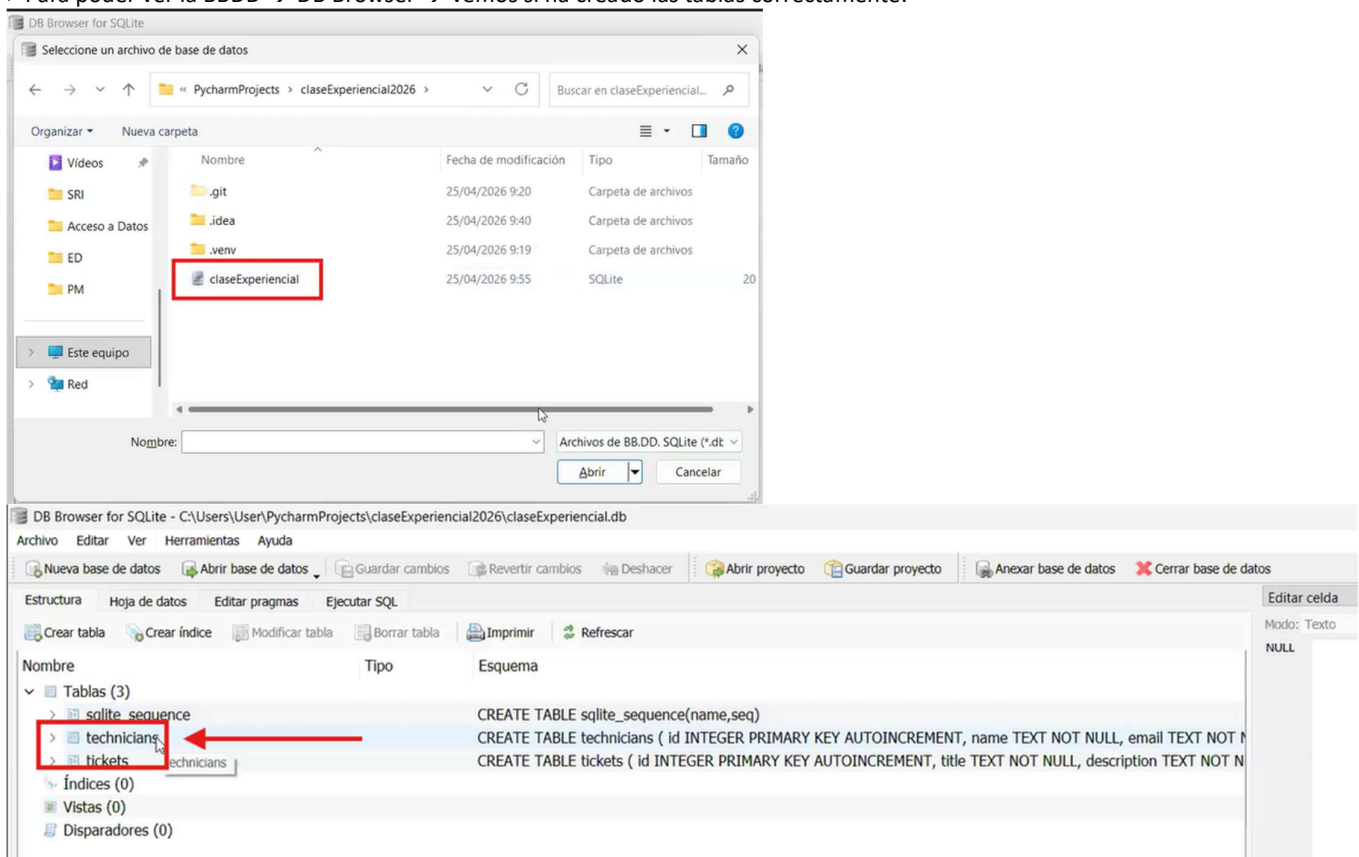
    cursor.execute("""
    CREATE TABLE tickets (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        description TEXT NOT NULL,
        created_by TEXT NOT NULL,
        status TEXT NOT NULL,
        technician_id INTEGER,
        FOREIGN KEY (technician_id) REFERENCES technicians(id)
    ) """)

    conn.commit()
    conn.close()
```

```
if __name__ == "__main__":
    init_db()
```

→ Con esto iniciamos la BBDD → C del CRUD.

> Para poder ver la BBDD → DB Browser → vemos si ha creado las tablas correctamente:



> Vamos a tener un CRUD para la tabla de técnicos y uno para la de tickets.

> Ahora creamos el repositorio →

```
app.py × db.py models.py repository.py × service.py seed.py
8 from models import Technician
9
10
11 class TechnicianRepository:
12     def create(self, technician: Technician):
13         conn = get_connection()
14         cursor = conn.cursor()
15
16         cursor.execute("""
17             INSERT INTO technician (name, email, active) VALUES (?, ?, ?),
18             (technician.name, technician.email, 1 id technician.active else 0)
19         """)
20
21         conn.commit()
22         technician_id = cursor.lastrowid #la última fila del cursor (al ser autoincremental el id más grande)
23         conn.close()
24
25         return Technician(
26             id = technician_id,
27             name = technician.name,
28             email = technician.email,
29             active = technician.active
30         )
31
```

> Una función que me devuelva una lista con todos los técnicos:

```
app.py db.py models.py repository.py ×
1 from models import Technician, Ticket
2 from db import get_connection
3 from typing import List
4
```

```
app.py db.py models.py repository.py × service.py seed.py
13 class TechnicianRepository:
14     def create(self, technician: Technician):
15
16         return Technician(
17             id = technician_id,
18             name = technician.name,
19             email = technician.email,
20             active = technician.active
21         )
22
23
24
25
26
27
28
29
30
31
32
33
34 def find_all(self) -> List[Technician]:
35     conn = get_connection()
36     cursor = conn.cursor()
37     cursor.execute("SELECT * FROM technician")
38     rows = cursor.fetchall() #fetchone recoge una / fetchall recoge todas las filas
39     conn.close()
```

En rows tengo todos los ids de la tabla.

```
app.py  db.py  models.py  repository.py ×
from models import Technician, Ticket
from db import get_connection
from typing import List, Optional
```

Optional ¿¿¿¿

```
def find_by_id(self, technician_id: int) -> Optional[Technician]:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("""
        SELECT * FROM technician WHERE technician_id = ?,
        (technician.id)
    """)
    row = cursor.fetchone()
    conn.close()
```

```
app.py  db.py  models.py  repository.py ×
from models import Technician, Ticket
from db import get_connection
from typing import List, Optional
from sqlite3 import Row
```

```
def map_row_to_technician(self, row: Row) -> Technician:
    return Technician(
        id = row["id"],
        name = row["name"],
        email = row["email"],
        active = bool(row["active"])
    )
```

> En el repositorio va metiendo todas las funcionalidades conforme las necesita según desarrolla la app.

```
def find_all(self) -> List[Technician]:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM technician")
    rows = cursor.fetchall() #fetchone recoge una / fetchall recoge todas las filas
    conn.close()
    return [self.map_row_to_technician(row) for row in rows]

def find_by_id(self, technician_id: int) -> Optional[Technician]:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("""
        SELECT * FROM technician WHERE technician_id = ?,
        (technician.id)
    """)
    row = cursor.fetchone()
    if row is None:
        return None
    conn.close()
    return self.map_row_to_technician(row)
```

```
app.py db.py models.py repository.py x service.py
1 from models import Technician, Ticket, TicketStatus
2 from db import get_connection
3 from typing import List, Optional
4 from sqlite3 import Row
```

```
class TicketRepository:
    def create(self, ticket: Ticket):
        conn = get_connection()
        cursor = conn.cursor()

        cursor.execute("""
            INSERT INTO tickets (title, description, created_by, status, technician_id) VALUES (?, ?, ?, ?, ?),
            (ticket.id, ticket.description, ticket.created_by, ticket.status.value, ticket.technician_id)
        """)

        conn.commit()
        ticket_id = cursor.lastrowid #la última fila del cursor (al ser autoincremental el id más grande)
        conn.close()

        return Ticket(
            id = ticket_id,
            title = ticket.title,
            description = ticket.description,
            created_by = ticket.created_by,
            status = ticket.status,
            technician_id = ticket.technician_id
        )
    ]
```

```
def find_all(self) -> List[Ticket]:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM tickets")
    rows = cursor.fetchall() #fetchone recoge una / fetchall recoge todas las filas
    conn.close()
    return [self.map_row_to_tickets(row) for row in rows]

def find_by_id(self, ticket_id: int) -> Optional[Ticket]:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("""
        SELECT * FROM tickets WHERE id = ?,
        (ticket.id)
    """)
    row = cursor.fetchone()
    if row is None:
        return None
    conn.close()
    return self.map_row_to_ticket(row)
```

> Añade 3 métodos: uno para actualizar el estado, otro para asignar un técnico y otro para encontrar los tickets abiertos.

```
def update_status(self, ticket_id: int, status: TicketStatus) -> bool:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("""
        UPDATE tickets SET status = ? WHERE id = ?,
        (status.value, ticket_id)
    """)
    conn.commit()
    update = cursor.rowcount > 0
    return update
```

Update devuelve True si encuentra para actualizar y si no hay ninguno para actualizar devuelve False.

```
def assign_tecnicia(self, ticket_id: int, technician_id: int) -> bool:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(
        sql: "UPDATE tickets SET technician_id = ?, status = ? WHERE id = ?",
        parameters: (technician_id, TicketStatus.IN_PROGRESS.value, ticket_id)
    )
    conn.commit()
    update = cursor.rowcount > 0
    conn.close()
    return update
```

```
def find_open_tickets(self) -> List[Ticket]:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(sql: """
        SELECT *
        FROM tickets
        WHERE status = ?
    """, parameters: (TicketStatus.OPEN.value,))
    rows = cursor.fetchall()
    conn.close()

    return [self.map_row_to_ticket(row) for row in rows]
```

Ya tengo en el repositorio todas las operaciones que quiero hacer en la BBDD para ir llamándolas cuando necesite.

> service.py:

> Por un lado se define el CRUD (SQL).

> En service las operaciones que se quieren hacer (empresa privada).

```
app.py db.py models.py repository.py service.py x seed.py

1 from repository import TechnicianRepository, TicketRepository
2 class Service:
3     def __init__(self):
4         self.technician_repository = TechnicianRepository()
5         self.ticket_repository = TicketRepository()
```



```

def register_technician(self, name: str, email: str) -> TechnicianRepository:
    technician = Technician(
        id = None,
        name = name,
        email = email
    )
    return self.technician_repository.create(technician)

def create_ticket(self, title: str, description: str, created_by: str) -> Ticket:
    ticket = Ticket(
        id = None,
        title = title,
        description = description,
        created_by = created_by,
        status = Ticket.Status.OPEN,
        technician_id = None
    )
    return self.ticket_repository.create(ticket)

```

```

def assign_ticket(self, ticket_id: int, technician_id: int) -> None:
    ticket = self.ticket_repository.find_by_id(ticket_id)
    if ticket is None:
        raise ValueError("El ticket {ticket_id} no existe")
    technician = self.technician_repository.find_by_id(technician_id)
    if technician is None:
        raise ValueError(f"El técnico {technician_id} no existe")
    if not technician.active:
        raise ValueError(f"El técnico no está activo")
    if ticket.status == TicketStatus.CLOSED:
        raise ValueError(f"No se puede asignar un ticket cerrado")

    update = self.ticket_repository.assign_technician(ticket.id, technician_id)
    if not update:
        raise ValueError("No se ha podido asignar un ticket")

```

```

def close_ticket(self, ticket_id: int) -> None:
    ticket = self.ticket_repository.find_by_id(ticket_id)
    if ticket is None:
        raise ValueError(f"El ticket {ticket_id} no existe")
    if ticket.status == TicketStatus.CLOSED:
        raise ValueError(f"El ticket ya se encuentra cerrado")

    update = self.ticket_repository.update_status(ticket.id, TicketStatus.CLOSED)
    if not update:
        raise ValueError("No se pudo cerrar el ticket")

```

 app.py
  db.py
  models.py
  repository.py
  service.py 
 seed.py

```

1 from models import Technician, Ticket, TicketStatus
2 from repository import TechnicianRepository, TicketRepository
3 from typing import List, Optional
4

```

```

def get_all_tickets(self) -> List[Ticket]:
    return self.ticket_repository.find_all()

def get_all_technicians(self) -> List[Technician]:
    return self.technician_repository.find_all()

def get_open_tickets(self) -> List[Ticket]:
    return self.ticket_repository.find_open_tickets()

def get_ticket_by_id(self, ticket_id: int) -> Optional[Ticket]:
    return self.ticket_repository.find_by_id(ticket_id)

```

> APP:

```
app.py x db.py models.py
1 from service import Service
2
3 def print_separator() -> None:
4     print("\n" + "=" * 60 + "\n")
5
```

Una función separadora que no devuelve nada, solo imprime por pantalla salto de línea y 60 veces el carácter *

```
app.py x db.py models.py repository.py
1 from service import Service
2
3 def print_separator() -> None: 1 usage
4     print("\n" + "=" * 60 + "\n")
5
6 def main()->None: 1 usage
7     service = Service()
8
9     print_separator()
10
11 if __name__ == "__main__":
12     main()
```

```
C:\Users\User\PycharmProjects\claseExperiencial2026\.venv\Scripts\python.exe C:\Users\User\PycharmProjects\claseExperiencial2026\app.py

=====

Process finished with exit code 0
```

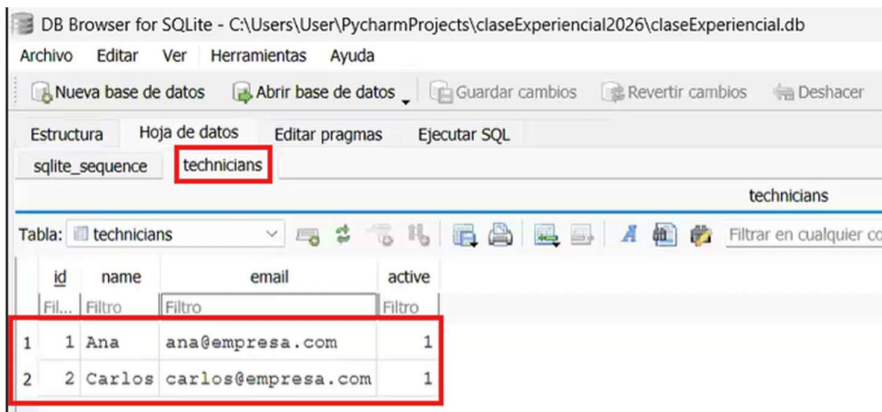
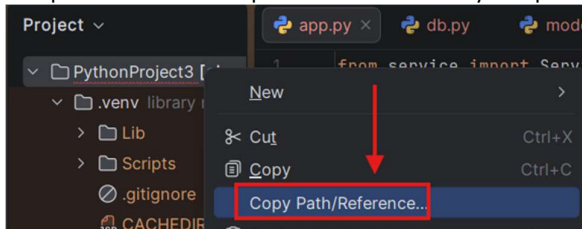
```
app.py x db.py models.py repository.py service.py seed.py
1 from service import Service
2
3 def print_separator() -> None: 1 usage
4     print("\n" + "=" * 60 + "\n")
5
6 def main()->None: 1 usage
7     service = Service()
8
9     print_separator()
10    print("1. Registrando técnicos...")
11    tech1 = service.register_technician(name="Ana", email="ana@empresa.com")
12    tech2 = service.register_technician(name="Carlos", email="carlos@empresa.com")
13
14
15 if __name__ == "__main__":
16     main()
```

```
=====

1. Registrando técnicos...

Process finished with exit code 0
```


> Copia la ruta absoluta para abrir en la BBDD y comprobar que ha funcionado:



> Vamos a ir llamando a todos los métodos de service.py:

```
def main()->None: 1 usage
    service = Service()

    print_separator()
    print("2. Creando tickets...")
    ticket1 = service.create_ticket(
        title: "VPN no funciona",
        description: "El empleado no pudo conectarse",
        created_by: "Juan"
    )
    ticket2 = service.create_ticket(
        title: "Portátil muy lento",
        description: "El equipo tarda",
        created_by: "María"
    )

if __name__ == "__main__":
    main()
```

```
=====

2. Creando tickets...

Process finished with exit code 0
```

DB Browser for SQLite - C:\Users\User\PycharmProjects\claseExperiencial2026\claseExperiencial.db

Archivo Editar Ver Herramientas Ayuda

Nueva base de datos Abrir base de datos Guardar cambios Revertir cambios Des

Estructura Hoja de datos Editar pragmas Ejecutar SQL

sqlite_sequence technicians tickets tickets

tickets

Tabla: tickets

	id	title	description	created_by	status	technician_id
	Fil...	Filtro	Filtro	Filtro	Filtro	Filtro
1	1	VP...	El ...	Juan	OPEN	NULL
2	2	Portá...	El ...	Maria	OPEN	NULL

```
def main()->None: 1 usage
    service = Service()
    print("1. Registrando técnico")
    tech3 = service.register_technician( name= "Antonio", email= "antonio@empresa.com")

    print_separator()
    print("2. Creando tickets...")
    ticket3 = service.create_ticket(
        title= "VPN no funciona",
        description= "El empleado no pudo conectarse",
        created_by= "Antonio"
    )
    ticket2 = service.create_ticket(
        title= "Portátil muy lento",
        description= "El equipo tarda",
        created_by= "María"
    )
    print_separator()
    print("3. Asignando el ticket 3 al técnico 3...")
    service.assign_ticket(ticket3.id, tech3.id)

    print_separator()
    print("4. Cerrando el ticket 3...")
    service.close_ticket(ticket3.id)

if __name__ == "__main__":
    main()
```

```
def map_row_to_technician(self, row: Row) -> Technician:
    return Technician(
        id = row[0],
        name = row[1],
        email = row[2],
        active = bool(row[3])
    )
```

```
def map_row_to_ticket(self, row: Row) -> Ticket: 2 usages
    return Ticket(
        id = row[0],
        title = row[1],
        description = row[2],
        created_by = row[3],
        status = TicketStatus(row[4]),
        technician_id = row[5]
    )
```

DB Browser for SQLite - C:\Users\User\PycharmProjects\claseExperiencial2026\claseExperiencial.db

Archivo Editar Ver Herramientas Ayuda

Nueva base de datos Abrir base de datos Guardar cambios Revertir cambios Deshacer Abrir proyecto

Estructura Hoja de datos Editar pragmas Ejecutar SQL

sqlite_sequence tickets

tickets

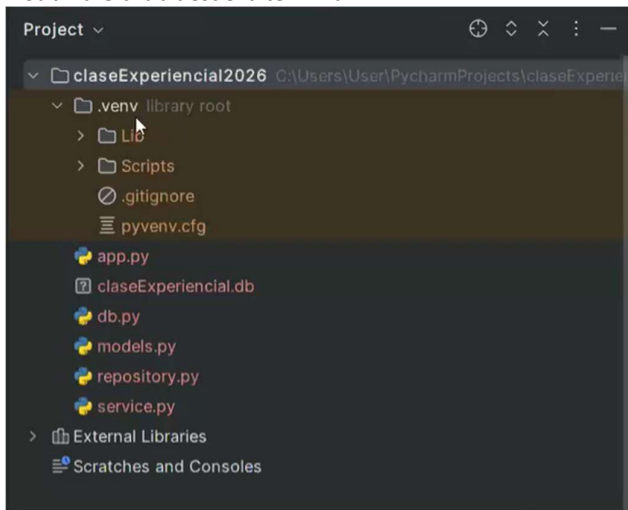
Tabla: tickets

Filtrar en cualquier columna

	id	title	description	created_by	status	technician_id
	Fil...	Filtro	Filtro	Filtro	Filtro	Filtro
1	1	VPN no funciona	El empleado no pudo conectarse	juan	CLOSED	1
2	2	Portatil muy lento	El equipo tarda	Maria	OPEN	NULL
3	3	VPN no funciona	El empleado no pudo conectarse	antonio	OPEN	NULL

> Semilla no da tiempo.

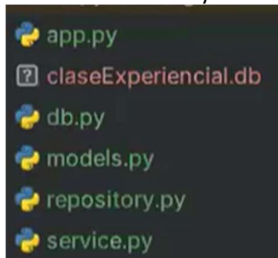
> Subir a Github desde la terminal.



> Lo que hay en venv no queremos que se suba.

```
git add app.py db.py models.py repository.py service.py
```

> Pasan a verde → ya están añadidos al repositorio (aún no se ha hecho una copia):



```
(.venv) PS C:\Users\User\PycharmProjects\claseExperiencial2026> git commit -m "Primer commit"
[master (root-commit) 1b0a6d6] Primer commit
5 files changed, 304 insertions(+)
create mode 100644 app.py
create mode 100644 db.py
create mode 100644 models.py
create mode 100644 repository.py
create mode 100644 service.py
(.venv) PS C:\Users\User\PycharmProjects\claseExperiencial2026>
```

> Ahora si ha hecho una copia de cada archivo en la rama Master.

> Vamos a Github:

General

Owner *  ajcarmona99

Repository name *

✓ IncidenciaTickets is available.

> Le da a crear:

Quick setup — if you've done this kind of thing before

Set up in Desktop or HTTPS SSH <https://github.com/ajcarmona99/IncidenciaTickets.git>

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# IncidenciaTickets" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/ajcarmona99/IncidenciaTickets.git
git push -u origin main
```

...or push an existing repository from the command line

```
git remote add origin https://github.com/ajcarmona99/IncidenciaTickets.git
git branch -M main
git push -u origin main
```

> Copiamos ese comando y lo ejecutamos en la terminal de PyCharm:

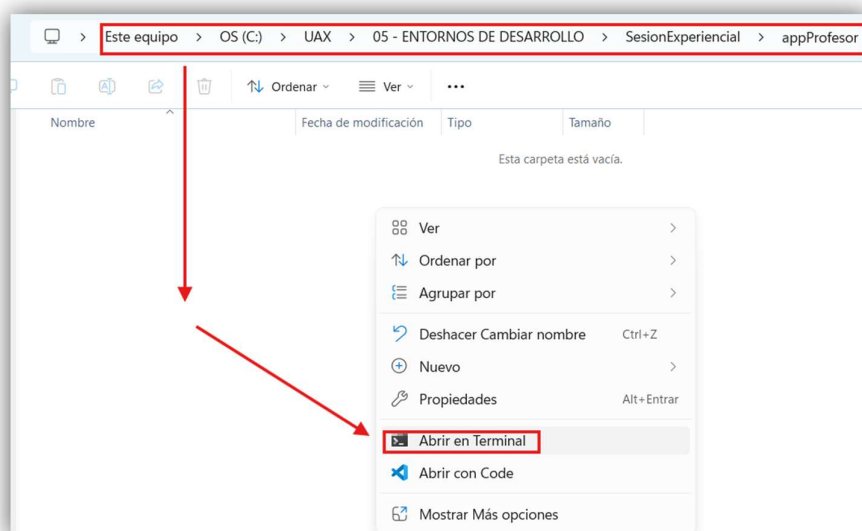
```
git remote add origin https://github.com/ajcarmona99/IncidenciaTickets.git
```

Desde ahora, en local cuando ponga **origin** me estará refiriendo a ese enlace del repositorio en GitHub, lo he añadido como origin.

> Ahora → subo el commit que he hecho a GitHub:

```
(.venv) PS C:\Users\User\PycharmProjects\claseExperiencial2026> git push -u origin master
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 16 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 2.70 KiB | 1.35 MiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/ajcarmona99/IncidenciaTickets.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
(.venv) PS C:\Users\User\PycharmProjects\claseExperiencial2026>
```

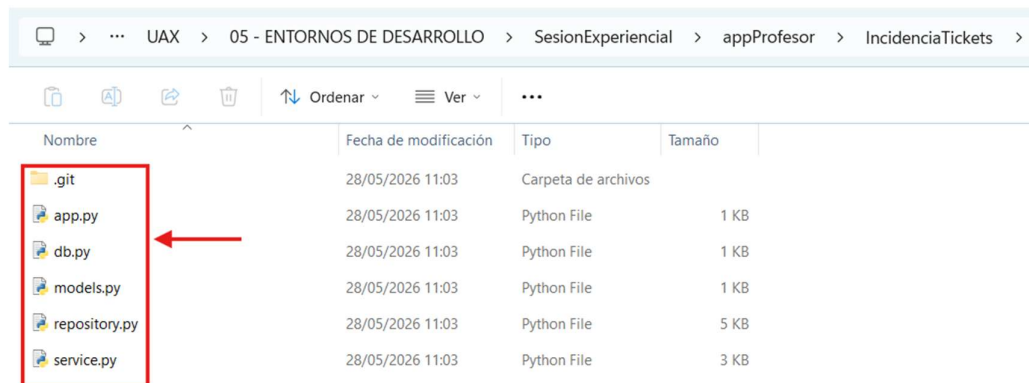
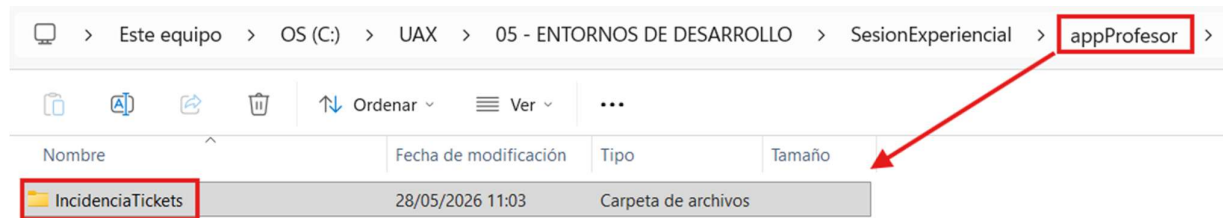
> Para descargarnos su repositorio → en la carpeta en la que quiero bajarlo → abro una terminal:



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\UAX\05 - ENTORNOS DE DESARROLLO\SesionExperiencial\appProfesor> git clone https://github.com/ajcarmona99/IncidenciaTickets
Cloning into 'IncidenciaTickets'...
remote: Enumerating objects: 7, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 7 (delta 0), reused 7 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (7/7), done.
PS C:\UAX\05 - ENTORNOS DE DESARROLLO\SesionExperiencial\appProfesor> |
```



qué hace enum de Enum en python?
En Python, Enum (que se importa desde el módulo enum) sirve para crear enumeraciones.
En programación, una enumeración es un conjunto de nombres simbólicos vinculados a valores únicos y constantes. Piensa en ellos como una forma de crear tus propios tipos de datos para cosas que tienen opciones fijas (como los días de la semana, los meses, los colores del semáforo o los estados de un pedido).
Aquí tienes el desglose de qué hace, por qué se usa y cómo funciona.
¿Cómo se usa? (Ejemplo básico)
Para usarlo, creas una clase que hereda de Enum:
<pre>from enum import Enum class EstadoPedido(Enum): PENDIENTE = 1 PROCESANDO = 2 ENVIADO = 3 ENTREGADO = 4</pre>
Una vez definido, puedes usarlo en tu código así:
<pre># Asignar a una variable pedido_actual = EstadoPedido.PROCESANDO # Comparar de forma segura if pedido_actual == EstadoPedido.PROCESANDO: print("El almacén está preparando tu paquete.")</pre>
¿Por qué deberías usar Enum en lugar de variables comunes o strings?
Si no usaras Enum, probablemente usarías strings ("pendiente", "enviado") o números enteros (1, 2). Esto trae problemas que Enum resuelve elegantemente:
• Evita errores de dedo (Type safety): Si escribes mal un string (por ejemplo, "enbiado" con 'b'), tu programa fallará silenciosamente o se comportará raro. Con Enum, si escribes EstadoPedido.ENVIADO incorrectamente, Python te dará un error inmediatamente. • Código autodocumentado: Cualquiera que lea tu código sabrá exactamente cuáles son las únicas opciones válidas para esa variable. • Inmutabilidad: Los valores de un Enum son constantes. Nadie puede cambiar el valor de EstadoPedido.PENDIENTE a mitad de la ejecución del programa. • Legibilidad y limpieza: En lugar de recordar si el número 3 significaba "Enviado" o "Cancelado", usas un nombre claro.
Características clave que debes conocer
Cada miembro de un Enum tiene dos propiedades automáticas: .name (el texto del nombre) y .value (el valor asignado).
<pre>print(EstadoPedido.ENVIADO.name) # Resultado: 'ENVIADO' print(EstadoPedido.ENVIADO.value) # Resultado: 3</pre>
Además, los enums son iterables, lo que significa que puedes hacerles un bucle para listar todas las opciones:
<pre>for estado in EstadoPedido: print(f"Código {estado.value} significa {estado.name}")</pre>
Un truco extra: Si no te importan los números que tienen asociados y solo quieres que sean únicos, Python te ofrece auto() para no tener que escribir los números a mano:
<pre>from enum import Enum, auto class Color(Enum): ROJO = auto() VERDE = auto() AZUL = auto()</pre>
En este código que me pasas "from enum import Enum"
<pre>class EstadoPedido(Enum): PENDIENTE = 1 PROCESANDO = 2 ENVIADO = 3 ENTREGADO = 4" no entiendo la utilidad de 1, 2 ... No le das uso posterior</pre>
Es una duda totalmente lógica. En el ejemplo corto que te di, parece que esos números están ahí de adorno porque solo usamos el nombre (EstadoPedido.PROCESANDO).
Sin embargo, esos números (los valores) son fundamentales tras bambalinas. Aquí te explico las tres razones principales de por qué están ahí y para qué se usan en el "mundo real":
1. Guardar en Bases de Datos o enviar en APIs (Eficiencia)
En el código usas los nombres porque son fáciles de leer para ti, pero a las bases de datos o a los sistemas externos les gusta más guardar números porque ocupan mucho menos espacio y son más rápidos de procesar.

En la Base de Datos: Guardas un simple 3.
Al leerlo en Python: Conviertes ese 3 de vuelta en algo legible usando el valor:
<pre># Imagina que recibes un '3' de tu base de datos o de una API externa valor_desde_bd = 3 # Lo transformas instantáneamente en el Enum de Python estado = EstadoPedido(valor_desde_bd) print(estado) # Resultado: EstadoPedido.ENVIADO</pre>
2. Ordenar y Comparar (Lógica de negocio) Al asignarles números, puedes usar operadores lógicos como "mayor que" (>) o "menor que" (<) si heredas de clases como IntEnum. Por ejemplo, puedes verificar si un pedido ya pasó por la etapa de preparación de forma muy sencilla si los números van en orden cronológico: <pre># Si PENDIENTE es 1 y PROCESANDO es 2... # Un número mayor o igual a 2 significa que el pedido ya se empezó a trabajar. if pedido_actual.value >= EstadoPedido.PROCESANDO.value: print("Ya no puedes cancelar el pedido, está en preparación o enviado.")</pre>
<pre># Si PENDIENTE es 1 y PROCESANDO es 2... # Un número mayor o igual a 2 significa que el pedido ya se empezó a trabajar. if pedido_actual.value >= EstadoPedido.PROCESANDO.value: print("Ya no puedes cancelar el pedido, está en preparación o enviado.")</pre>
3. Es un requisito de Python (Cada miembro necesita una identidad) La sintaxis de Python exige que a toda variable dentro de una clase se le asigne algo. No puedes dejar PENDIENTE vacío. Si los números te molestan o realmente no te importa qué valor tengan porque no los vas a guardar en ningún lado, puedes usar strings o la función auto() que te mencioné antes: <pre>from enum import Enum, auto class EstadoPedido(Enum): PENDIENTE = auto() # Python le asigna el 1 automáticamente PROCESANDO = auto() # Python le asigna el 2 automáticamente</pre>
O incluso strings, si prefieres que en la base de datos se guarde texto en lugar de números:
<pre>class EstadoPedido(Enum): PENDIENTE = "PEN" PROCESANDO = "PRO"</pre>
En resumen: Tú en tu código interactúas con las palabras (EstadoPedido.PROCESANDO), pero Python y tus sistemas de almacenamiento interactúan con los valores (2). Es un puente entre el lenguaje humano y el lenguaje de la máquina.